

Эссе: Multi-stage build для компилируемых языков в Docker

В современном DevOps и контейнеризации часто возникает задача: **собирать приложения на компилируемых языках** (Java, Go, C#) внутри Docker, но при этом финальный образ должен быть **легким и безопасным**. Для компиляции требуется «тяжёлый» SDK (Software Development Kit), включающий компиляторы, библиотеки и инструменты сборки, а для запуска достаточно «runtime» — минимальной среды выполнения, которая умеет только исполнять уже скомпилированный код.

Проблема:

Если использовать один образ для сборки и запуска, финальный контейнер будет включать весь SDK. Это приводит к:

- большому размеру образа (несколько сотен мегабайт вместо десятков),
- потенциальным уязвимостям, потому что в контейнере есть лишние инструменты,
- усложнению поддержки и обновления.

Решение: multi-stage build в Docker.

1. Конструкция FROM ... AS builder

В Dockerfile можно объявить несколько этапов сборки. Первый этап называется «builder»:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
WORKDIR /app
```

```
COPY *.csproj ./
```

```
RUN dotnet restore
```

```
COPY . ./
```

```
RUN dotnet publish -c Release -o /app/publish
```

- Здесь мы используем **SDK-образ**, который включает все инструменты сборки.
- В этом этапе мы компилируем приложение (dotnet publish) и получаем **готовые к запуску артефакты** (DLL, JAR, бинарники Go) в папке /app/publish.
- Этот этап полностью изолирован, и всё, что не скопировано в следующий этап, останется только в образе builder.

2. Конструкция COPY --from=builder

На втором этапе мы используем **легкий runtime-образ**:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
```

```
WORKDIR /app
```

```
COPY --from=build /app/publish ./
```

```
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

- **COPY --from=build** позволяет **взять только нужные файлы из предыдущего этапа**.

- Мы копируем только скомпилированное приложение, без SDK, исходников и инструментов сборки.
- Финальный образ становится компактным, безопасным и готовым к продакшену.

3. Преимущества подхода

1. **Минимальный размер образа:** в финальный контейнер попадают только runtime и скомпилированные артефакты.
2. **Безопасность:** отсутствуют компиляторы и SDK, что снижает риск атак.
3. **Разделение ответственности:** один этап отвечает за сборку, другой — за исполнение.
4. **Повторяемость и чистота:** build-артефакты можно собирать в отдельном, полностью контролируемом окружении, не загрязняя финальный образ.

Заключение

Multi-stage build в Docker — это эффективный способ разделить **сборку и выполнение**. Конструкции FROM ... AS builder и COPY --from=builder решают проблему тяжёлых SDK и позволяют создавать **компактные, безопасные и воспроизводимые контейнеры** для компилируемых языков. Этот подход стал стандартом в индустрии DevOps, особенно при работе с Java, Go и C#, где размер SDK значительно больше, чем размер скомпилированного приложения.